

Studienarbeit

Projektarbeit T3000

aus dem Studiengang Informationstechnik

an der Dualen Hochschule Baden-Württemberg Ravensburg Campus
Friedrichshafen

von

Finn Ludwig, Ekaterina Karavaeva, David Pelcz, Alexander
Bernecker

08.07.2026

Inhaltsverzeichnis

1. Einleitung	1
1.1. Motivation und Kontext	1
1.2. Problemstellung und Forschungsfrage	1
1.3. Zielsetzung und Abgrenzung	2
1.4. Aufbau der Arbeit	3
2. Theoretische Grundlagen	3
2.1. Maschinelles Lernen und Reinforcement Learning	3
2.2. Proximal Policy Optimization (PPO)	4
2.3. Sequenzmodellierung für RL	5
2.3.1. Long Short-Term Memory (LSTM)	5
2.3.2. Transformer-Architektur	5
2.4. Wahrnehmung in RL-Agenten	5
2.5. Unity ML-Agents Toolkit	6
3. Stand der Technik	6
4. Methodik	7
4.1. Wissenschaftliche Rahmung	7
4.2. Agenten-Matrix und Vergleichsdesign	8
4.3. Kontrollierte Variablen	8
4.4. Evaluationsprotokoll	9
4.4.1. Primärmetriken	9
4.4.2. Generalisierungsmetriken	9
4.4.3. Statistische Auswertung	9
5. Systemarchitektur	10
5.1. Gesamtüberblick	10
5.2. Map-System	10
5.2.1. Datenmodell	10
5.2.2. MapGenerator (Runtime)	10
5.2.3. Prozedurale Map-Generierung	11
5.3. Agent-System	11
5.3.1. Aktionsraum	11
5.3.2. Observation-Space	12
5.3.3. Bewegungs- und Sprungphysik	12
5.3.4. Third-Person-Kamera	12
5.4. Sensorik	12

5.5.	Reward-System	13
5.6.	Trainingsinfrastruktur	13
6.	Modellarchitekturen	14
6.1.	MLP-Baseline	14
6.2.	LSTM-Memory (Custom Policy)	14
6.3.	Transformer-Memory (Custom Policy)	14
6.3.1.	Architektur	14
6.3.2.	Rolling-Memory-Buffer (Fix v2)	15
6.3.3.	Integration in ML-Agents	15
6.4.	Geplante Architekturen (M8–M10)	15
6.4.1.	CNN-Encoder (Nature-CNN)	15
6.4.2.	Multi-Sensor Late Fusion	15
7.	Umsetzung	16
7.1.	Milestones 1–2: Map-System und Mehrere Layouts	16
7.2.	Milestone 3: Agent-Grundsystem	16
7.3.	Milestone 4: Hindernisse und Todeslogik	16
7.4.	Milestone 5: Reward-System und erstes Training	17
7.5.	Milestone 6: Prozedurale Generierung und Curriculum	17
7.6.	Milestone 7: Transformer- und LSTM-Integration	17
7.6.1.	Iterative Fehlerdiagnose und Fixes	18
7.6.2.	Trainings-Iterationen V5–V13	18
7.6.3.	V14: Vollständiger Curriculum-Durchlauf und Lava-Diagnose	19
7.7.	Direkter Architekturvergleich: MLP vs. LSTM vs. Transformer	20
8.	Evaluation	21
8.1.	MLP-Baseline (Milestone 5)	21
8.2.	Transformer- und LSTM-Iterationen (Milestone 7)	21
8.3.	Paarweise Architektur-Vergleiche	22
8.3.1.	Ray-Gruppe: LSTM (A3) vs. Transformer (A4)	22
8.3.2.	Kamera-Gruppe: LSTM (A1) vs. Transformer (A2)	22
8.3.3.	Multi-Sensor-Gruppe: LSTM (A6) vs. Transformer (A5)	22
8.4.	Generalisierung auf held-out Maps	22
8.5.	Statistische Auswertung	23
8.6.	Diskussion	23
9.	Übertragbarkeit und praktische Anwendbarkeit	23
9.1.	Analogie Labyrinth und reale Navigationsszenarien	23
9.2.	Hardware- und Software-Anforderungen	24
9.3.	Bewertungsmatrix	24
9.4.	Limitationen der Übertragbarkeit	24

10. Fazit und Ausblick	25
10.1. Zusammenfassung	25
10.2. Beantwortung der Forschungsfragen	25
10.3. Limitationen und Lessons Learned	25
10.4. Ausblick	26
Anhang	27
11. Trainingskonfiguration (Direktvergleich)	27
12. Reward-Tabelle (Stand V13)	28
13. Curriculum-Phasen (Direktvergleich, Stand Juli 2026)	28
14. Bearbeitete GitHub-Issues (Übersicht)	29
15. Hardware- und Software-Stack	29
16. Repository-Struktur	29
Literatur	31

Abbildungsverzeichnis

Abbildung 1 Einordnung der verwendeten Konzepte: Reinforcement Learning mit neuronalem Netz, trainiert per Actor-Critic mit PPO	4
--	----------

Tabellenverzeichnis

Tabelle 1	Abbildung des Labyrinth-Szenarios auf reale Navigationsprobleme	1
Tabelle 2	Agenten-Matrix des Vergleichsdesigns	8
Tabelle 3	Primärmetriken des Evaluationsprotokolls	9
Tabelle 4	Trainings-Iterationen V5–V13 (Transformer) mit Kerndiagnose	19
Tabelle 5	V14: Phasen-Performance über 31,2 Mio. Steps	20
Tabelle 6	Hardware-Anforderungen je Sensor-Konfiguration	24

Abkürzungsverzeichnis

AI	Artificial Intelligence
BFS	Breadth-First Search
CNN	Convolutional Neural Network
GAE	Generalized Advantage Estimation
IT	Informationstechnologie
KI	Künstliche Intelligenz
LSTM	Long Short-Term Memory
MDP	Markov Decision Process
ML	Machine Learning
MLP	Multi-Layer Perceptron
ONNX	Open Neural Network Exchange
PBRS	Potential-Based Reward Shaping
PPO	Proximal Policy Optimization
RL	Reinforcement Learning

1. Einleitung

1.1. Motivation und Kontext

Reinforcement Learning (RL) hat sich in den vergangenen Jahren als leistungsfähiger Ansatz für autonome Navigationsaufgaben etabliert – von Servicerobotern in Restaurants und Krankenhäusern über Lager- und Indoor-Logistik bis hin zu nicht-spielergesteuerten Charakteren in Spielen und Simulationen. Allen diesen Anwendungen ist gemeinsam, dass ein Agent in einer teilweise unbekannten, veränderlichen Umgebung ein Ziel erreichen muss, ohne dass ihm der Weg dorthin explizit vorgegeben wird.

Die vorliegende Arbeit verwendet ein selbst entwickeltes 3D-Labyrinth in der Unity-Engine als kontrollierte Abstraktion solcher realen Navigationsprobleme. Die Elemente des Labyrinths lassen sich direkt auf reale Szenarien abbilden:

Labyrinth-Element	Reales Szenario
Korridore & Wände	Gänge in Restaurants, Lagern, Krankenhäusern
Lava / Löcher	Treppen, nasse Böden, Schächte, Kabel
Prozedurales Layout	Täglich verändertes Umgebungslayout (umgeräumte Möbel)
Goal-Navigation	Tisch anfahren (Kellnerroboter), Regal ansteuern (Logistik)
Episodenende	Abgeschlossene Lieferfahrt

Tabelle 1 – Abbildung des Labyrinth-Szenarios auf reale Navigationsprobleme

Im Gegensatz zu klassischen Pathfinding-Problemen muss der Agent nicht nur einen Weg finden, sondern situativ auf unterschiedliche Gefahren reagieren: springen, umkehren, Hindernisse umgehen. Die Forschungslücke, die diese Arbeit adressiert, liegt in der Kombination zweier Fragen: Wie gut generalisiert ein RL-Agent über prozedural generierte, nie gesehene Map-Layouts – und welchen Einfluss hat dabei die Wahl der Netzwerkarchitektur, insbesondere der Vergleich zwischen Transformer-basierten [1] und rekurrenten (LSTM, [2]) Sequenzmodellen im RL-Kontext?

1.2. Problemstellung und Forschungsfrage

Die zentrale Forschungsfrage der Arbeit lautet:

Kann ein Transformer-basierter RL-Agent in einer selbst gebauten 3D-Labyrinthwelt generalisierbares Navigations- und Hindernisvermeidungsverhalten erlernen, das sich auf unbekannte Map-Layouts übertragen lässt?

Im Forschungsplan für die Milestones 6–11 wurde diese Hauptfrage in vier erweiterte Forschungsfragen ausdifferenziert:

- **RQ1 — Sensortyp:** Welchen Einfluss hat der Sensortyp (Ray / Kamera / kombiniert) auf Lerngeschwindigkeit und Erfolgsrate in einem prozeduralen Labyrinth-Navigationsproblem?
- **RQ2 — Temporal-Architektur:** Wie unterscheiden sich LSTM und Transformer in ihrer Fähigkeit, temporale Abhängigkeiten bei unterschiedlichen Sensoreingaben zu nutzen?
- **RQ3 — Sensor-Fusion:** Verbessert die Kombination mehrerer Sensortypen die Robustheit und Generalisierung auf unbekannten Maps im Vergleich zu Einzelsensor-Agenten?
- **RQ4 — Praktische Übertragbarkeit:** Welche Sensor-Architektur-Konfiguration eignet sich unter Berücksichtigung von Leistung, Hardwareaufwand und Latenz am besten für den Transfer auf reale Navigationsprobleme?

Aus diesen Fragen wurden vier Hypothesen abgeleitet: H1 (Multi-Sensor-Agenten generalisieren besser als Einzelsensor-Agenten), H2 (Transformer übertreffen LSTM stärker bei Kamera- als bei Ray-Observations), H3 (Ray-only-Agenten konvergieren schneller, generalisieren aber schlechter) und H4 (die Kombination CNN + Transformer mit kombiniertem Sensor ist die robusteste Gesamtkonfiguration).

1.3. Zielsetzung und Abgrenzung

Der Pflichtumfang der Arbeit umfasst: eine selbst gebaute 3D-Labyrinthwelt in Unity mit grid-basiertem Map-System und mindestens fünf unterschiedlichen Map-Layouts, einen Agenten mit Bewegung in vier Richtungen und physikbasierter Sprungmechanik, ray-basierte Sensorik zur Unterscheidung von Wänden, Hindernistypen und Ziel, die Basis-Hindernisse Lava (tödlich, überspringbar), Löcher (tödlich, nicht passierbar) und Sackgassen, ein Transformer-basiertes Entscheidungsmodell als Kernmodell, eine MLP-Baseline zum direkten Vergleich, ein Reward-System, mindestens eine Ablationsstudie mit Generalisierungstest auf unbekannten Maps, Beobachtbarkeit über TensorBoard-Metriken, ein reproduzierbares Git-Repository sowie den vorliegenden schriftlichen Bericht.

Über den Pflichtumfang hinaus wurden folgende optionale Erweiterungen umgesetzt: eine vollständig prozedurale Map-Generierung, Curriculum Learning mit gestaffelten Schwierigkeitsphasen, ein zusätzlicher LSTM-Vergleichsagent sowie Multi-Area- und Multi-Instanz-Training zur Parallelisierung.

Nicht Gegenstand dieser Arbeit sind der Sim-to-Real-Transfer auf physische Roboter, Multi-Agent-Setups sowie dynamische (bewegliche) Hindernisse.

1.4. Aufbau der Arbeit

Kapitel 2 legt die theoretischen Grundlagen von Reinforcement Learning, PPO, LSTM- und Transformer-Architekturen sowie des Unity-ML-Agents-Toolkits. Kapitel 3 ordnet die Arbeit in den Stand der Technik ein. Kapitel 4 beschreibt das methodische Vorgehen und das experimentelle Design. Kapitel 5 dokumentiert die Systemarchitektur der Trainingsumgebung, Kapitel 6 die implementierten Modellarchitekturen. Kapitel 7 zeichnet die Umsetzung entlang der Projekt-Milestones nach, einschließlich der iterativen Fehlerdiagnose. Kapitel 8 präsentiert die bisherigen Evaluationsergebnisse, Kapitel 9 diskutiert die Übertragbarkeit auf reale Anwendungen, und Kapitel 10 zieht ein Fazit.

2. Theoretische Grundlagen

2.1. Maschinelles Lernen und Reinforcement Learning

Reinforcement Learning ist ein Teilgebiet des maschinellen Lernens (ML), das wiederum ein Teilgebiet der künstlichen Intelligenz (KI) ist. Im Gegensatz zum überwachten Lernen (Lernen aus gelabelten Beispielen) und zum unüberwachten Lernen (Strukturentdeckung in ungelabelten Daten) lernt ein RL-Agent durch Interaktion mit einer Umgebung: Er führt Aktionen aus, beobachtet deren Konsequenzen und erhält ein skalaras Belohnungssignal (Reward) [3].

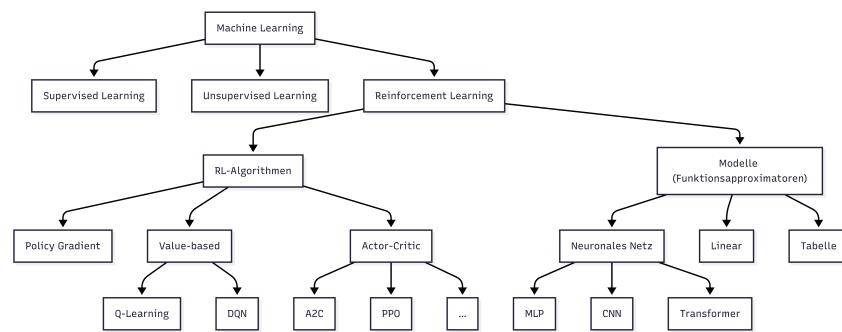


Abbildung 1 – Einordnung der verwendeten Konzepte: Reinforcement Learning mit neuronalem Netz, trainiert per Actor-Critic mit PPO

Formal wird das Problem als Markov-Entscheidungsprozess (MDP) modelliert: eine Menge von Zuständen S , Aktionen A , einer Übergangsfunktion $P(s'|s, a)$, einer Reward-Funktion $R(s, a)$ und einem Discount-Faktor $\gamma \in [0, 1]$, der zukünftige Belohnungen gegenüber unmittelbaren abwertet. Der Agent lernt eine Policy $\pi(a|s)$ – eine Wahrscheinlichkeitsverteilung über Aktionen gegeben einen Zustand. Die Value-Funktion $V^\pi(s)$ schätzt den erwarteten kumulierten, diskontierten Reward ab Zustand s ; die Q-Funktion $Q^\pi(s, a)$ analog für Zustands-Aktions-Paare. On-Policy-Verfahren (wie das hier eingesetzte PPO) lernen aus Daten, die mit der aktuellen Policy gesammelt wurden, während Off-Policy-Verfahren auch ältere oder fremde Daten wiederverwenden können.

2.2. Proximal Policy Optimization (PPO)

Das Training aller Agenten dieser Arbeit erfolgt mit Proximal Policy Optimization [4] im Actor-Critic-Framework: Das Policy-Netz (Actor) entscheidet, welche Aktion ausgeführt wird, das Value-Netz (Critic) bewertet, wie gut der erreichte Zustand ist. Der Critic liefert damit ein Baseline-Signal, das die Varianz des Policy-Gradienten reduziert – der Advantage ergibt sich als Differenz zwischen tatsächlichem und erwartetem Reward.

Das Kernproblem reiner Policy-Gradient-Verfahren sind zu große Policy-Updates: Der Agent „vergisst“, was zuvor funktioniert hat, und das Training destabilisiert sich. PPO begrenzt deshalb das Verhältnis zwischen neuer und alter Policy durch einen Clipping-Mechanismus (in dieser Arbeit durchgehend $\varepsilon = 0.2$). Zur Advantage-Schätzung wird Generalized Advantage Estimation (GAE) mit $\lambda = 0.95$ verwendet, was einen Kompromiss zwischen Bias und Varianz der Schätzung darstellt. Gegenüber Vanilla Policy Gradient bietet PPO damit deutlich stabileres Training bei vergleichbarer Recheneffizienz – ein Grund, weshalb es der Standard-Algorithmus des ML-Agents-Toolkits ist.

2.3. Sequenzmodellierung für RL

Ein rein reaktiver Agent (MLP) trifft jede Entscheidung ausschließlich auf Basis der aktuellen Observation. Für Navigationsaufgaben mit teilweiser Beobachtbarkeit — der Agent sieht immer nur einen Ausschnitt des Labyrinths — ist jedoch zeitlicher Kontext hilfreich: „Ich laufe seit mehreren Schritten auf dieselbe Wand zu,“ ist eine Information, die erst über eine Sequenz von Observations entsteht. Hierfür existieren zwei etablierte Architekturfamilien.

2.3.1. Long Short-Term Memory (LSTM)

Das LSTM [2] verarbeitet Sequenzen rekurrent über einen internen Zellzustand, der durch drei Gates (Forget, Input, Output) selektiv Information über Zeitschritte weitergibt. Der Vorteil im RL-Kontext ist das implizite Gedächtnis: Es muss kein expliziter Sequenz-Buffer verwaltet werden, der Hidden State trägt die Historie. Im ML-Agents-Toolkit ist ein LSTM standardmäßig über die memory-Einstellung der Netzwerkkonfiguration verfügbar. Nachteile sind die sequenzielle (nicht parallelisierbare) Verarbeitung über Zeitschritte und begrenzte Langzeitabhängigkeiten.

2.3.2. Transformer-Architektur

Der Transformer [1] ersetzt Rekurrenz durch Self-Attention: Jede Position einer Sequenz kann direkt auf jede andere Position attendieren, Multi-Head-Attention erlaubt mehrere parallele Aufmerksamkeits-Muster, und Positional Encodings kodieren die Reihenfolge. Im RL-Kontext wurde die Architektur unter anderem durch den Decision Transformer [5] und GTrXL [6] etabliert. Gegenüber dem LSTM bietet der Transformer direkten Zugriff auf alle Zeitschritte des Kontextfensters und parallele Verarbeitung im Training; dem stehen ein deutlich höherer Parameterbedarf, quadratische Komplexität in der Sequenzlänge und — wie sich in dieser Arbeit zeigen wird — nicht-triviale Konsistenzanforderungen zwischen Inferenz und Training unter PPO gegenüber.

2.4. Wahrnehmung in RL-Agenten

ML-Agents stellt mehrere Sensortypen bereit, die sich in Informationsgehalt und Rechenaufwand unterscheiden: Der **VectorSensor** liefert manuell definierte numerische Zustandswerte (Position, Geschwindigkeit, Statusflags) und eignet sich für klassische RL-Aufgaben. Der **RayPerceptionSensor** sendet mehrere Raycasts aus und kodiert, welcher Objekttyp in welcher Distanz getroffen wird — der Standardansatz für Navigation und Hinderniserkennung.

Der **CameraSensor** liefert Pixel-Tensoren für vision-basierte Aufgaben und erfordert einen CNN-Encoder. Daneben existieren **GridSensor** (diskretes Raster) und **BufferSensor** (variable Objektlisten). Für stabile Lernverläufe ist die Normalisierung der Beobachtungsräume wesentlich; in dieser Arbeit werden alle manuellen Observations auf Wertebereiche um $[-1.5, 1]$ skaliert.

2.5. Unity ML-Agents Toolkit

Das ML-Agents-Toolkit [7] verbindet eine Unity-Umgebung mit einem Python-Trainer über eine gRPC-Kommunikationsschicht. Auf Unity-Seite besteht ein Agent aus der Agent-Klasse (mit den Methoden `CollectObservations()`, `OnActionReceived()` und `OnEpisodeBegin()`), den Behavior Parameters (Definition von Observation- und Action-Space), einem Decision Requester (Entscheidungsfrequenz) und Sensor-Komponenten. Auf Python-Seite steuert eine YAML-Konfigurationsdatei sämtliche Hyperparameter; trainierte Modelle werden ins ONNX-Format exportiert und können direkt in Unity zur Inferenz geladen werden. Trainingsmetriken werden kontinuierlich nach TensorBoard geschrieben. Das Projekt verwendet mlagents in der Version 0.30.0 (Python) mit dem Unity-Package 2.0.2; im späteren Projektverlauf (direkter Architekturvergleich) kam mlagents 1.1.0 mit PyTorch 2.3.1 zum Einsatz.

3. Stand der Technik

Klassische Pathfinding-Algorithmen wie A* oder Dijkstra lösen Navigationsprobleme optimal, setzen jedoch vollständiges Wissen über die Karte voraus und reagieren nicht auf lokale Wahrnehmung. RL-basierte Navigation kehrt diese Voraussetzung um: Der Agent kennt die Karte nicht und muss aus lokalen Sensordaten eine generalisierende Strategie lernen.

Als Meilenstein der RL-Forschung gilt das Deep-Q-Network von [8], das Atari-Spiele direkt aus Pixeln auf menschlichem Niveau lernte und dessen CNN-Architektur („Nature-CNN“) bis heute als Referenz-Encoder für visuelle RL-Observations dient. Für verkörperte Navigation (Embodied AI) existieren Simulationsplattformen wie Habitat und AI2-THOR, für autonomes Fahren CARLA. Speicher-erweiterte RL-Architekturen reichen vom Differentiable Neural Computer bis zu GTrXL [6], das gezielt die Instabilität von Transformern im RL-Training

adressiert; der Decision Transformer [5] formuliert RL vollständig als Sequenzmodellierungsproblem.

Zwei weitere Forschungsstränge sind für diese Arbeit direkt relevant: Curriculum Learning [9] – das Training mit ansteigender Aufgabenschwierigkeit – bildet das Rückgrat der hier verwendeten Trainingsphasen, und prozedurale Level-Generierung als Mittel gegen Overfitting in RL wurde von [10] systematisch untersucht: Agenten, die auf einer festen Menge von Levels trainieren, memorieren diese, statt zu generalisieren – prozedural generierte Trainingslevels erzwingen übertragbare Strategien. Ergänzend kommt in dieser Arbeit potentialbasiertes Reward Shaping [11] (nachweislich policy-invariant) und Curiosity-getriebene Exploration [12] zum Einsatz.

4. Methodik

4.1. Wissenschaftliche Rahmung

Die in Kapitel 1 formulierten Forschungsfragen RQ1–RQ4 werden über ein kontrolliertes Vergleichsexperiment beantwortet. Die zugehörigen Hypothesen lauten im Detail:

- **H1 (Multi-Sensor-Vorteil):** Multi-Sensor-Agenten erzielen höhere Erfolgsraten auf held-out Maps als Einzelsensor-Agenten, weil sie redundante und komplementäre Information aus zwei Modalitäten nutzen können.
- **H2 (Transformer bei Kamera):** Transformer-Agenten übertreffen LSTM-Agenten stärker bei Kamera-Observations als bei Ray-Observations, da Pixel-Sequenzen komplexere zeitliche Strukturen aufweisen, von denen der Attention-Mechanismus stärker profitiert.
- **H3 (Ray-Konvergenz):** Ray-only-Agenten konvergieren schneller als Kamera-Agenten, generalisieren jedoch schlechter auf prozedural generierte Maps.
- **H4 (Beste Gesamtkonfiguration):** CNN+Transformer mit kombiniertem Sensor erzielt die höchste Erfolgsrate auf held-out Maps.

4.2. Agenten-Matrix und Vergleichsdesign

Das vollständige Experiment-Design sieht sieben Agenten-Konfigurationen vor, die sich in genau einer bzw. zwei kontrollierten Variablen (Sensortyp, Temporal-Architektur) unterscheiden:

Agent	Sensor	Visual Encoder	Temporal	Vergleichsgruppe
Baseline	Ray (VectorSensor)	—	MLP	Referenz (M5)
A3	Ray	—	LSTM	Ray-Gruppe
A4	Ray	—	Transformer	Ray-Gruppe
A1	Kamera	CNN	LSTM	Kamera-Gruppe
A2	Kamera	CNN	Transformer	Kamera-Gruppe
A6	Kamera + Ray	CNN	LSTM	Multi-Sensor
A5	Kamera + Ray	CNN	Transformer	Multi-Sensor

Tabelle 2 – Agenten-Matrix des Vergleichsdesigns

Die paarweisen Vergleiche folgen dem Prinzip „nur eine Variable ändert sich,,: Architekturvergleiche innerhalb jeder Sensorgruppe (A3 vs. A4, A1 vs. A2, A6 vs. A5) beantworten RQ2; der Vergleich der jeweils besten Konfiguration pro Gruppe beantwortet RQ1 und RQ3; der Vergleich aller Agenten gegen die MLP-Baseline quantifiziert den Mehrwert temporaler Architekturen. Die Ray-Gruppe wird zuerst bearbeitet, da sie auf der bestehenden Sensor-Infrastruktur aufbaut; Kamera- und Multi-Sensor-Gruppen folgen in späteren Milestones.

4.3. Kontrollierte Variablen

Um Konfundierung zu vermeiden, werden alle nicht untersuchten Parameter über alle Agenten identisch gehalten: das Trainingsschritt-Budget, die Reward-Struktur, der Map-Generierungsalgorithmus samt Curriculum, der RL-Algorithmus (PPO) mit seinen Kernhyperparametern (Learning Rate $3 \cdot 10^{-4}$, $\gamma, \varepsilon = 0.2$, $\lambda = 0.95$) sowie die Seeds für unabhängige Wiederholungsläufe (geplant: 42, 123, 456, 789, 1337). Im aktuell laufenden direkten Architekturvergleich (Kapitel 7) trainieren alle drei Ray-Agenten (MLP, LSTM, Transformer) zusätzlich **gleichzeitig im selben Environment-Prozess** — sie durchlaufen damit exakt dieselben Curriculum-Phasen und Map-Sequenzen, was Umgebungsunterschiede als Störvariable vollständig eliminiert.

4.4. Evaluationsprotokoll

4.4.1. Primärmetriken

Metrik	Definition
Erfolgsrate	Anteil Episoden mit Goal-Erreichen (Rolling-Fenster, TensorBoard SuccessRate)
Konvergenzgeschwindigkeit	Steps bis erstmaliges Erreichen von 80 % Erfolgsrate
Kollisionsrate	Anteil Episoden, die durch Lava-/Hole-Tod enden (DeathByLava, DeathByHole)
Mean Episodenlänge	Durchschnittliche Schritte pro Episode
Cumulative Reward	TensorBoard-Kurve Environment/Cumulative Reward

Tabelle 3 – Primärmetriken des Evaluationsprotokolls

Ergänzend werden projektspezifische Diagnostik-Metriken geloggt: Todesursache (Lava/Hole/Timeout), Distanz zum Ziel bei Timeout, Lava-Sprungversuche und erfolgreiche Lava-Überquerungen sowie die aktuelle Curriculum-Phase.

4.4.2. Generalisierungsmetriken

Nach Abschluss des Trainings werden alle Agenten auf held-out Maps evaluiert, die während des Trainings nie geladen wurden. Geplant sind drei fixe Evaluations-Maps aus unterschiedlichen Generierungsalgorithmen (Recursive Backtracking, Cellular Automata, Room-Placement). Als Kennzahl dient neben der Generalisierungs-Erfolgsrate der **Overfitting-Index**: die Differenz zwischen Trainings- und Generalisierungs-Erfolgsrate.

4.4.3. Statistische Auswertung

Da RL-Rewards nicht normalverteilt sind, ist als Signifikanztest der nicht-parametrische Mann-Whitney-U-Test vorgesehen, mit Bonferroni-Korrektur für multiple Vergleiche (bei zehn paarweisen Tests $\alpha' = 0.005$). Als Effektgröße dient Cliff's Delta, ergänzt um 95%-Bootstrap-Konfidenzintervalle auf allen aggregierten Metriken. Lernkurven werden mit Konfidenzband über die Seeds visualisiert.

TODO Die statistische Auswertung setzt abgeschlossene Mehrfach-Läufe (5 Seeds) voraus und steht noch aus – der aktuelle Vergleichslauf (Kapitel 8) ist ein Einzellauf mit Seed 42.

5. Systemarchitektur

5.1. Gesamtüberblick

Das Gesamtsystem besteht aus drei Komponenten: der Unity-Umgebung (im Editor oder als Headless-Standalone-Build), dem Python-Trainer (`mlagents-learn`) und TensorBoard zur Beobachtung. Unity simuliert die Labyrinthwelt und den Agenten, sammelt Observations und führt die vom Trainer gelieferten Aktionen aus; der Trainer optimiert die Policy per PPO und exportiert Checkpoints als ONNX-Modelle.

Das Code-Layout im Repository trennt die Verantwortlichkeiten klar: `Assets/Scripts/Map/` enthält das Map-System (Datenmodell, Generator, prozedurale Generierung, Curriculum), `Assets/Scripts/Agent/` die Agenten-Logik, `Assets/Scripts/Camera/` die Kamerasteuerung, `config/` die versionierten YAML-Trainingskonfigurationen, `training/` Python-Hilfsskripte (`venv-Patch`, Analyse) und `results/` die Trainingsartefakte. Die Funktionslogik liegt vollständig im Code; der Unity-Inspector dient ausschließlich der Konfiguration und Verdrahtung von Komponenten.

5.2. Map-System

5.2.1. Datenmodell

Kern des Map-Systems ist das `CellType-Enum` mit den Werten `Empty`, `Floor`, `Wall`, `Obstacle`, `Goal`, `SpawnPoint` sowie den später ergänzten Typen `Lava` (betretbar, tödlich, bis Tiefe 1 überspringbar), `Hole` (nicht betretbar, nicht überspringbar) und `Platform` (schwebendes Bodenfeld 0,75 Einheiten über Lava). Eine Map wird als `MapData-ScriptableObject` gespeichert: ein flaches eindimensionales Array von `CellTypes` mit Breite und Höhe, adressiert über $\text{index} = y \cdot \text{width} + x$.

5.2.2. MapGenerator (Runtime)

Der `MapGenerator (MonoBehaviour)` instanziiert zur Laufzeit für jede Zelle das passende Prefab an der Weltposition $\text{mapRoot} + (x \cdot \text{cellSize}, 0, y \cdot \text{cellSize})$ mit $\text{cellSize} = 1$. Spawnpunkt, Ziel und Hindernisse werden dynamisch pro Episode platziert; konfigurierbare Modi (`SpawnPlacementMode`, `GoalPlacementMode`, `ObstaclePlacementMode`) erlauben wahlweise zufällige Platzierung auf Floor-Zellen oder vordefinierte Marker-Zellen aus dem Layout.

Eine BFS-Pfadvalidierung stellt vor jedem Episodenstart sicher, dass ein begehbarer Pfad vom Spawn zum Ziel existiert. Der MapSelectionMode unterstützt feste, zufällige, sequenzielle und curriculum-gesteuerte Layoutwahl. Für Trainingsdurchsatz sorgen ein Multi-Area-Setup (mehrere parallele, räumlich versetzte Trainingsbereiche in einer Szene) sowie ein TilePool, der Map-Tiles wiederverwendet statt sie pro Episode zu zerstören und neu zu instanziiieren.

5.2.3. Prozedurale Map-Generierung

Die prozedurale Generierung erzeugt vollständige MapData-Assets ohne manuellen Designer-Input über die Pipeline BuildTopology → PlaceRooms → PlaceCorridors → PlaceWalls → PlaceSpawnAndGoal → CoverageCheck → PlaceClusters → PlacePlatforms → PathValidation. Der RoomCorridorGraph baut eine Topologie aus Start-, Goal- und Sackgassen-Räumen mit 2 Tiles breiten Korridoren, rekursiver Verzweigung (max. Tiefe 5, max. 28 Räume) und einer festen 2-Tile-Pufferzone (BORDER) an allen Rändern. Terminal-Korridore sind echte Sackgassen, die mit einem Hole-Cluster enden. Der ObstacleClusterPlacer platziert Lava- und Hole-Cluster nach einem Regelwerk pro Korridortyp (z. B. immer genau ein Lava-Cluster auf dem Goal-Korridor), der PlatformPlacer setzt bei tiefen Lava-Clustern Plattformen. Der SemanticPathfinder validiert abschließend die Lösbarkeit per BFS mit semantischen Regeln: Lava der Tiefe 1 gilt als überspringbar, tiefere Lava nur mit Plattform, Holes nie.

Die Schwierigkeitsgrade sind als Enum gestaffelt: Trivial (7×7 offener Raum) → TrivialCorr (zwei Räume, ein Korridor) → TrivialBranch (+ Ast-Korridor) → TrivialHole (+ 2×2-Hole am Ast-Ende) → TrivialHazard (+ Lava im Hauptkorridor) → Easy → Medium → Hard (bis 37×45 Tiles, hohe Verzweigung); später kam TrivialLava (flacher Raum mit einer 1-Zellen-Lava-Wand quer, die einen Sprung erzwingt) hinzu. Jedes Layout ist über einen Seed deterministisch reproduzierbar.

5.3. Agent-System

5.3.1. Aktionsraum

Der Aktionsraum besteht aus drei diskreten Branches: Bewegung (idle / vorwärts / rückwärts), Rotation (idle / links / rechts) und Sprung (nein / ja). Diese agent-relative Steuerung wurde im V11-Refactoring eingeführt: Zuvor bewegte sich der Agent in Weltkoordinaten, während Sen-

sorik und Zielvektor agent-relativ waren — die Policy musste die Koordinatentransformation implizit mitlernen. Seit V11 liegen Sensorik, Zielvektor und Aktionen im selben Bezugssystem.

5.3.2. Observation-Space

Der manuelle VectorSensor umfasst 14 Gleitkommawerte: die Eigengeschwindigkeit in lokalen Koordinaten (3), den Bodenkontakt-Status isGrounded (1), die Zielrichtung in lokalen Koordinaten (3), den Boden-Sensor (3 Raycasts $\times$ 2 Werte = 6) sowie — seit V12 als Unterstützung des Reward Shapings — die normalisierte Distanz zum Ziel (1). Hinzu kommt der automatisch verwaltete RayPerceptionSensor3D (11 Rays $\times$ 2 gestapelte Frames $\times$ (6 Tags + 2 Werte)).

5.3.3. Bewegungs- und Sprungphysik

Die Bewegung ist Rigidbody-basiert (MovePosition / MoveRotation / AddForce). Die Sprungkraft ist so kalibriert, dass eine 1-Zellen-Lücke überspringbar ist, eine 2-Zellen-Lücke nicht — Lava der Tiefe 1 ist damit per Sprung überwindbar, tiefere Hindernisse erzwingen Umwege oder Plattformen. Zwei Physik-Schutzmechanismen wurden im Projektverlauf notwendig: ein **Wall-Climb-Guard** (Bestrafung bei Überschreiten einer Maximalhöhe über dem Spawn-Niveau) und ein **Velocity-Cap** (maxUpwardVelocity), da die PhysX-Depenetration den Agenten beim Anlaufen gegen Wände unkontrolliert nach oben schleudern konnte.

5.3.4. Third-Person-Kamera

Für Beobachtung und Demos folgt eine ThirdPersonCamera dem Agenten: Die Zielposition wird im lokalen Agenten-Raum berechnet (hinter und über dem Agenten) und in LateUpdate() per SmoothDamp geglättet, die Rotation per Slerp. LateUpdate() garantiert, dass die Kamera erst nach der Agentenbewegung aktualisiert wird.

5.4. Sensorik

Der horizontale RayPerceptionSensor3D ist mit 11 Rays über 120° konfiguriert (menschähnliches Sichtfeld, deckt seitliche Abzweigungen ab), Ray-Länge 12 Zellen, Sphere-Cast-Radius 0.25 (angepasst an den Kapsel-Radius des Agenten) und Stacked Raycasts = 2 für implizite Bewegungserkennung über den Frame-Vergleich. Die sechs erkennbaren Tags sind Wall, Obstacle, Lava, Hole, Goal und Bridge.

Da die Ray-Komponente ausschließlich horizontal castet, wurde zusätzlich ein manueller **Boden-Sensor** implementiert: drei Raycasts nach unten (unter dem Agenten, eine und zwei

Zellen in Bewegungsrichtung voraus), die jeweils einen Typ-Code und die normalisierte Distanz liefern. Die Typ-Codes unterscheiden Floor (+1.0), Bridge (+0.5), Hole (−0.5), Lava (−1.0) und echten Abgrund ohne Treffer (−1.5).

5.5. Reward-System

Die Reward-Funktion setzt sich aus terminalen Ereignissen, einer Zeitstrafe und Shaping-Termen zusammen:

$$R = r_{\text{goal}} \cdot \mathbb{1}[\text{Ziel}] + r_{\text{lava}} \cdot \mathbb{1}[\text{Lava}] + r_{\text{hole}} \cdot \mathbb{1}[\text{Hole}] + r_{\text{timeout}} \cdot \mathbb{1}[\text{Timeout}] + r_{\text{step}} \cdot T + F_{\text{PBRs}}$$

Die Werte wurden über die Trainingsiterationen (Kapitel 7) mehrfach kalibriert; der Stand der Iteration V13 beträgt: `goalReward = +30`, `lavaDeathPenalty` und `holeDeathPenalty = −3`, `timeoutPenalty = −10` (Timeout ist damit bewusst schlechter als ein aktiver, gescheiterter Versuch), `stepPenalty = −0.001` pro Schritt, mit curriculum-abhängigem `MaxStep` pro Phase. Das Potential-Based Reward Shaping [11] liefert dichtes Feedback über die Zielannäherung, $F = (\text{prevDist} - \gamma \cdot \text{currDist}) \cdot \text{scale}$, und ist nachweislich policy-invariant. Seit V13 ergänzt ein Curiosity-Modul [12] (strength 0.05) die extrinsische Belohnung um intrinsische Exploration, um unerforschte Bereiche — insbesondere hinter Lava-Barrieren — attraktiv zu machen. Alle Reward-Werte sind als serialisierte Felder zentral am `LabyrinthAgent` konfiguriert; externe Trigger-Objekte lösen nur `OnTriggerEnter` aus, die Reward-Vergabe erfolgt ausschließlich in der Agent-Klasse (ML-Agents-Paradigma).

5.6. Trainingsinfrastruktur

Das Training läuft in einer Python-venv (`mlagents 0.30.0`, `PyTorch 2.0.1+cu118`; später `mlagents 1.1.0`, `PyTorch 2.3.1`). Da `mlagents 0.30.0` keine öffentliche Custom-Policy-API besitzt, werden die eigenen Speicher-Module über ein idempotentes Patch-Skript (`training/patch_mlagents.py`, mit `--undo-Option`) direkt in die venv eingebracht. Zur Parallelisierung wurden je nach Projektphase Multi-Area-Setups (4–16 Areas pro Unity-Instanz) und Headless-Parallelisierung eingesetzt — im Maximalausbau 6 Unity-Instanzen $\times$ 16 Areas = 96 parallele Agenten mit ca. 850–1050 Steps/s. Als Hardware kamen u. a. eine RTX 3050 Laptop-GPU mit Ryzen 5 5625U sowie ein Surface-Gerät (CPU-Training) zum Einsatz. Für unbeaufsichtigte Langzeitläufe sorgt ein Watchdog-Batch-Skript, das den Trainer bei Abstürzen automatisch mit `--resume` neu startet.

6. Modellarchitekturen

6.1. MLP-Baseline

Die Baseline ist das Standard-ML-Agents-Netz: zwei Hidden-Layer mit je 256 Units, angewendet direkt auf den konkatenierten Observation-Vektor, ohne jegliches Zeitgedächtnis. In Milestone 5 erreichte die Baseline `mlp_baseline_v2` nach 2 Mio. Steps mit 10 parallelen Agenten einen mittleren Reward von +0.81 (bei damaliger Reward-Skala mit `goalReward = +1`).

6.2. LSTM-Memory (Custom Policy)

Statt des Built-in-LSTM wurde ein eigenes `LSTMMemory`-Modul implementiert, um eine exakt vergleichbare Schnittstelle zum Transformer zu garantieren: gleicher Einbettungspunkt im `NetworkBody`, gleiche Eingabe (`[batch, seq_len, 256]` aus dem MLP-Encoder), gleiche GAE-kompatible Ausgabe-Form `[batch * seq_len, output_size]`. Das Modul besteht aus einem einschichtigen `nn.LSTM` mit `hidden_size = 64` ($= \text{memory_size} / 2$, mlagents-Konvention) und umfasst 82.432 Parameter. Die Integration erfolgt über additive `elif`-Blöcke im `NetworkBody`-Patch — die Transformer-Pfade bleiben unberührt.

6.3. Transformer-Memory (Custom Policy)

6.3.1. Architektur

Das `TransformerMemory`-Modul arbeitet auf `d_model = 256` ($= \text{hidden_units}$, kein zusätzlicher Embedding-Layer) mit `nhead = 4` (64 Dimensionen pro Head) und zwei Attention-Layern; es umfasst ca. 1.070.000 Parameter — rund das Dreizehnfache des LSTM-Moduls. Das Positional Encoding ist gelernt (`nn.Embedding`), die Positions-Indizes sind als `register_buffer` hinterlegt, damit der ONNX-Export funktioniert. Zwei PyTorch-2.0-spezifische Workarounds waren erforderlich: `nn.TransformerEncoderLayer` löste unter CUDA einen Segfault im optimierten C-Kernel aus und wurde durch manuelle `nn.MultiheadAttention + LayerNorm + Feed-Forward`-Blöcke ersetzt; `batch_first = True` verursachte einen bekannten ONNX-Export-Bug und wurde durch `batch_first = False` mit expliziten Transposes umgangen.

6.3.2. Rolling-Memory-Buffer (Fix v2)

Der schwerwiegendste konzeptionelle Bug der Transformer-Integration: ML-Agents ruft die Policy während der Inferenz mit `sequence_length = 1` auf, trainiert aber mit der YAML-Sequenzlänge (z. B. 8). Die PPO-Ratio $\pi_{\text{neu}}(\text{seq} = 8) / \pi_{\text{alt}}(\text{seq} = 1)$ war damit strukturell ungültig – nahezu jeder Gradient wurde geclippt, der Transformer lernte faktisch nichts. Ein LSTM ist davon nicht betroffen, da sein persistenter Hidden State Inferenz und Training deterministisch konsistent hält. Die Lösung: Der `memories`-Tensor wird als Rolling Buffer zweckentfremdet und speichert die letzten `seq_len - 1` MLP-Encodings; bei der Inferenz wird daraus die vollständige Sequenz rekonstruiert. Inferenz und Training sehen damit denselben Kontext, die Log-Wahrscheinlichkeiten sind konsistent und die PPO-Ratio gültig.

6.3.3. Integration in ML-Agents

Die Integration erfolgt als `venv`-Patch statt als Fork: Ein Fork von `mlagents` hätte dauerhafte Divergenz mit dem Upstream bedeutet, ein Frameworkwechsel (z. B. CleanRL) den Verlust der Unity-Integration. Das Patch-Skript ergänzt in `settings.py` das Feld `memory_type` (`lstm/transformer`), instanziiert im `NetworkBody` das jeweilige Modul und ist idempotent mit `--undo`-Option. Ein früher geplanter `BufferSensorComponent`-Ansatz auf Unity-Seite wurde verworfen, da `mlagents` den Replay-Buffer intern bereits in Sequenzen chunked – der Transformer erhält dieselbe Eingabeform wie das LSTM, ohne Unity-Code-Änderungen.

6.4. Geplante Architekturen (M8–M10)

6.4.1. CNN-Encoder (Nature-CNN)

Für die Kamera-Agenten ist die Nature-CNN nach [8] vorgesehen: drei Convolution-Layer ($32 \times 8 \times 8/4$, $64 \times 4 \times 4/2$, $64 \times 3 \times 3/1$) mit anschließendem Fully-Connected-Layer auf 256 Dimensionen, bei einer Eingabe von vier gestapelten Graustufen-Frames zu 84×84 Pixeln.

6.4.2. Multi-Sensor Late Fusion

Die Multi-Sensor-Agenten kombinieren CNN-Output (256) und VectorSensor (14) per Late Fusion: Beide Modalitäten werden per LayerNorm normalisiert, konkateniert und über einen Fully-Connected-Layer in einen gemeinsamen 256-dimensionalen Latent-Raum projiziert. Die LayerNorm vor der Konkatination ist kritisch, um zu verhindern, dass die hochdimensionale visuelle Modalität die Gradienten dominiert.

TODO CNN-Pfad und Multi-Sensor-Fusion sind geplant, aber noch nicht implementiert (Stand: Juli 2026). Der Projektfokus verschob sich zunächst auf den direkten Architekturvergleich in der Ray-Gruppe.

7. Umsetzung

7.1. Milestones 1–2: Map-System und Mehrere Layouts

In den ersten beiden Milestones entstand das Fundament: das Datenmodell (`CellType`, `MapData`, `MapGenerator`), fünf manuell entworfene Layouts (25×30 bzw. 18×30 Zellen) sowie ein Custom-Editor (`MapGeneratorEditor`) mit Preview-Buttons zum Generieren und Durchschalten der Layouts. Bereits hier wurde die dynamische Laufzeit-Platzierung eingeführt: Grundlayouts enthalten nur Wände und Böden; Spawnpunkt, Ziel und Hindernisse werden pro Episode zufällig auf gültigen Floor-Zellen platziert, abgesichert durch eine BFS-Pfadvalidierung (Issues #2, #3, #4, #19, #20, #39–#44).

7.2. Milestone 3: Agent-Grundsystem

Milestone 3 implementierte den `LabyrinthAgent` mit dem vollständigen ML-Agents-Lebenszyklus (`Initialize` / `OnEpisodeBegin` / `CollectObservations` / `OnActionReceived` / `Heuristic`), die Konfiguration des `RayPerceptionSensor3D` (11 Rays, 120°, 6 Tags), den manuellen Boden-Sensor sowie die Sprungkalibrierung: Mit `jumpForce = 3.5` und `moveSpeed = 2` ergibt sich eine Sprungweite von ca. 1,43 Zellen — eine 1-Zellen-Lücke ist überwindbar, eine 2-Zellen-Lücke nicht. Der Heuristic-Modus (WASD + Leertaste) diente der manuellen Validierung (Issues #21–#36).

7.3. Milestone 4: Hindernisse und Todeslogik

Für Lava und Löcher wurde bewusst ein tag-basierter Ansatz ohne neue `CellTypes` gewählt, da die dynamische Platzierung und die Sensorik bereits tag-basiert arbeiteten. Das Lava-Prefab ist eine flache Trigger-Platte mit kalibrierter Trigger-Höhe (Oberkante unterhalb der Sprunghöhe, sodass Springen den Trigger nicht auslöst, Hineinlaufen aber schon). Die Hole-Mechanik nutzt einen eigenen `HoleSurface`-Layer mit deaktivierter Kollision gegen den Default-Layer:

Der Agent fällt physisch durch das Loch und trifft eine unsichtbare KillZone-Box 20 Einheiten unter der Map. Die gesamte Todeslogik liegt zentral in `LabyrinthAgent.OnTriggerEnter` mit konfigurierbaren Penalties — ein verteilter DeathZone-Ansatz auf den Prefabs wurde als architektonisch inkonsistent verworfen (Issues #82–#88).

7.4. Milestone 5: Reward-System und erstes Training

Milestone 5 formalisierte die Reward-Strategie (`goalReward = +1`, `deathPenalty = -1`, `stepPenalty = -0.001`, `MaxStep = 2500` mit begründeter Überschlagsrechnung) und erstellte die erste versionierte YAML-Konfiguration (`config/labyrinth_training.yaml`). Ein Multi-Area-Setup mit vier parallelen Trainingsbereichen vervierfachte die Datensammlung; die TensorBoard-Integration wurde per Testlauf verifiziert. Die beiden ersten vollständigen Trainingsläufe: `mlp_baseline_v1` (CPU, 4 Agenten, manuell bei 600k Steps gestoppt, Reward +0.78) und `mlp_baseline_v2` (GPU, 10 Agenten, 2 Mio. Steps in ca. 1 h 44 min, finaler Reward +0.81). Eine wichtige, erst im Inference-Test entdeckte Limitierung: Der Parameter `Runtime Obstacles` stand auf 0 — das Modell trainierte ausschließlich auf hindernisfreien Maps und lernte reine Zielnavigation ohne Hindernisvermeidung (Issues #93–#99).

7.5. Milestone 6: Prozedurale Generierung und Curriculum

Milestone 6 ersetzte die fünf statischen Maps durch die vollautomatische prozedurale Generierung (Kapitel 5.2.3): variable Grid-Größen mit 2-Tile-Pufferzone, Terminal-Korridore als echte Sackgassen mit Hole-Abschluss, adaptive Korridorlängen, Coverage-Check und drei Schwierigkeitsgrade (Easy/Medium/Hard) über eine `DifficultySettings-Factory`. Parallel entstand das Curriculum-System: eine `CurriculumConfig` (`ScriptableObject`) definiert Phasen mit Layout-Listen und Episoden-Schwellenwerten, ein statischer `CurriculumTracker` verwaltet den Phasenfortschritt und liefert pro Episode das nächste Layout (Issues #127, #133, #134).

7.6. Milestone 7: Transformer- und LSTM-Integration

Milestone 7 integrierte die Custom-Transformer-Policy per `venv-Patch` (Issue #113) und die Custom-LSTM-Policy als direkte Baseline (Issue #131). Da sich die prozeduralen Easy-Maps für einen untrainierten Agenten als unlösbar herausstellten (Sparse-Reward-Problem, siehe unten), wurde die Trivial-Familie eingeführt — 7×7-Maps ohne Hindernisse sowie die

gestaffelten Varianten TrivialCorr, TrivialBranch, TrivialHole und TrivialHazard – und das Curriculum auf acht Phasen erweitert.

7.6.1. Iterative Fehlerdiagnose und Fixes

Fünf wesentliche Bugs wurden in dieser Phase identifiziert und behoben:

1. **PPO-Ratio-Inkonsistenz des Transformers** (Inferenz seq=1 vs. Training seq=8): behoben durch den Rolling-Memory-Buffer (Kapitel 6.3.2).
2. **Sparse Reward:** In über 400 Episoden auf prozeduralen Maps wurde das Ziel kein einziges Mal durch Zufallsexploration gefunden – ohne positives Signal existiert kein Gradient Richtung Ziel. Behoben durch die Trivial-Phase, PBRs-Shaping und die Distanz-Observation.
3. **Wall-Climb:** PhysX-Depenetration schleuderte den Agenten an Wänden nach oben. Behoben durch Wall-Climb-Guard und maxUpwardVelocity-Cap; eine erste Version des Guards beendete fälschlich Episoden nach 2–3 Steps und wurde entschärft.
4. **Eck-Heuristik:** Nur vier Trivial-Konfigurationen (seed % 4) führten dazu, dass der Agent „geh immer zu Position (5,5)“, memorierte statt dem Zielvektor zu folgen. Behoben durch zufällige Goal-Platzierung und ein Refactoring der Trivial-Generatoren mit randomisierten Raumgrößen, Korridorlängen und Grundformen.
5. **Goal-Collider halb im Boden:** Das Ziel wurde bei y=0 gespawnt, der Trigger ragte zur Hälfte unter den Boden. Behoben durch einen y-Offset von 0.5 in der Goal-Platzierung.

7.6.2. Trainings-Iterationen V5–V13

Der Weg zu einem lernfähigen Transformer verlief über neun dokumentierte Iterationen:

Version	Diagnose und Änderung
V5	Kein Causal Mask → bidirektionale Attention lernt eine bei Inferenz nicht existierende Funktion; kein Lernen
V6	Causal Mask + niedrigere Learning Rate; aber Beta (5e-3) zu hoch — Entropie-Regularisierung kämpft gegen Konvergenz
V7	Beta auf 1e-3: Entropy fällt deutlich (1.60), aber Value-Funktion kollabiert — Buffer (10k) zu klein für stabile Advantage-Schätzung
V8	Buffer auf 40.960, Time Horizon 256: stabilere, aber langsamere Updates
V9	8,7 Mio. Steps, Plateau bei Reward ≈ 8.9 und Episode Length 1199 (Dauer-Timeout): PBRS mit $\gamma = 0.99$ belohnte „Wandern“, statt Zielerreichen
V10	goalReward 10, stepPenalty -0.005, pbrsGamma 1.0 (nur echte Annäherung belohnt), Success-Rate-Logging eingeführt
V11	Agent-relative Bewegung + Dreh-Action (Action-Space 3/3/2), sequence_length 8 → 16; Erfolg 72–82 % auf einfachen Maps, aber Totalkollaps beim Phasenwechsel auf Lava-Maps
V12	timeoutPenalty -2 (Timeout schlechter als Tod), Lava-„Adrenalin-Reward“, mit Diminishing Returns, WallClimb-Guard 5.0; Overnight-Run mit 96 Agenten: Phasen 0–3 sauber, TrivialHazard nicht überwunden
V13	$\gamma = 0.997$ (Planungshorizont ca. 330 Steps), goalReward 30, timeoutPenalty -5, Lava-Attempt-Reward entfernt, PBRS-Scale 0.005, Curiosity-Modul, curriculum-abhängiges MaxStep (600–2000)

Tabelle 4 – Trainings-Iterationen V5–V13 (Transformer) mit Kerndiagnose

Der erste V13-Lauf war zusätzlich lehrreich als Konfigurations-Postmortem: Die C#-Quellcode-Änderungen waren nicht ins Agent-Prefab synchronisiert worden — der Build lief mit goalReward = 1 statt 30 und MaxStep = 6000 statt der phasenabhängigen Werte. Da Unity die Prefab-Serialisierung und nicht die Code-Defaults verwendet, war der Lauf eine unbeabsichtigte Hybrid-Konfiguration. Als Konsequenz wurde ein Verifikationsprotokoll etabliert: Nach jedem Build werden die ersten Minuten der Player-Logs auf die erwarteten Reward-Werte und MaxStep-Meldungen geprüft.

7.6.3. V14: Vollständiger Curriculum-Durchlauf und Lava-Diagnose

Der korrigierte Lauf V14 (31,2 Mio. Steps in 10 h 9 min, 96 parallele Agenten, 854 Steps/s) lieferte das bislang vollständigste Bild über alle acht Curriculum-Phasen:

Phase	Episoden	Erfolgsrate	Ø Reward
0 Trivial	7.110	65,5 %	-3,66
1 TrivialCorr	10.800	62,6 %	-4,17
2 TrivialBranch	15.600	78,4 %	-2,54
3 TrivialHole	21.600	79,4 %	-1,79
4 TrivialHazard	30.000	0,2 %	-0,95
5 Easy	48.000	3,5 %	-1,69
6 Medium	72.000	4,4 %	-1,64
7 Hard	109.129	21,0 %	-2,78

Tabelle 5 – V14: Phasen-Performance über 31,2 Mio. Steps

Zentrale Befunde: Die reine Navigation (Phasen 0–3) wurde gemeistert (Phase 0 in späten Buckets bis 100 %). Phase 4 (TrivialHazard) erwies sich als „totale Wand“, – 30.000 Episoden ohne Lerneffekt. Die Sprung-Statistik erklärt warum: Von 11.415 Lava-Sprüngen in Phase 4 endeten 99,96 % tödlich; in Phase 7 erfolgten 99,4 % aller Zielerreichungen gänzlich ohne Lava-Sprung. Der Agent hat nicht „Lava überspringen“ gelernt, sondern „Lava vollständig meiden“, – was auf großen Maps mit Umwegen funktioniert (21 % Erfolg in Hard), auf der erzwungenen Lava-Barriere von TrivialHazard aber scheitert. In Phase 7 zeigten sich zudem drei klare Crash-Recovery-Zyklen (Erfolgsrate 25 % → ca. 8 % → 25 %), ein Muster, das auf Catastrophic Forgetting bzw. PPO-Explorationsschwankungen hindeutet. Als Konsequenzen wurden für den Nachfolgelauf eine sanftere Hazard-Einführung, dichtere Sprung-Lernsignale und sauberere Sprung-Metriken (getrennte Zähler für Versuche und Erfolge) definiert.

7.7. Direkter Architekturvergleich: MLP vs. LSTM vs. Transformer

Auf Basis der V14-Erkenntnisse wurde das Experiment neu aufgesetzt (Branch ModelTrainingComparsion): Statt sequenzieller Einzelläufe trainieren drei Behaviors – MLP_Navigator, LSTM_Navigator und Transformer_Navigator – **gleichzeitig im selben Environment** mit identischem Curriculum, identischer Reward-Struktur und identischen PPO-Hyperparametern (Learning Rate $3e-4$, batch_size 2048, $\gamma = 0.995$, Curiosity 0.05, Ziel: 30 Mio. Steps pro Agent, Seed 42). Nur die memory-Einstellung unterscheidet sich: keine (MLP), LSTM (sequence_length 32, memory_size 256) bzw. Transformer (sequence_length 128, memory_size 256). Das Curriculum wurde auf sieben Phasen mit der neuen TrivialLava-Phase als sanfter Sprung-Lernstufe umgebaut: eine einzelne 1-Zellen-Lava-Wand quer durch einen flachen Raum erzwingt genau einen Sprung.

Das Training läuft als Headless-Standalone-Build mit zwei parallelen Environment-Instanzen und 20-facher Zeitskalierung, überwacht von einem Watchdog-Skript mit automatischem Resume und stündlich generierten Analyse-Reports. Zwei infrastrukturelle Probleme wurden während des Laufs diagnostiziert und behoben: (1) Der Curriculum-Fortschritt lag nur in statischen Feldern im Arbeitsspeicher und ging bei jedem Trainer-Neustart verloren – behoben durch atomare Persistierung des Curriculum-Zustands in JSON-Dateien pro Environment-Instanz. (2) Ein wiederkehrender Absturz im ca. 23-Minuten-Takt erwies sich als `MemoryError` des Python-Trainers beim gleichzeitigen Checkpoint-Export aller drei Behaviors (drei PPO-Buffer à 409.600 Steps überschritten den verfügbaren RAM) – behoben durch einheitliche Reduktion der `buffer_size` auf 102.400 für alle drei Agenten.

8. Evaluation

8.1. MLP-Baseline (Milestone 5)

Die MLP-Baseline `mlp_baseline_v2` (2 Mio. Steps, 10 Agenten, RTX 3050) konvergierte von -2.4 auf $+0.81$ mittleren Reward, mit dem Nulldurchgang bei ca. 220k Steps und weitgehender Sättigung ab ca. 1,4 Mio. Steps; die Trainingsdauer betrug ca. 1 h 44 min. Der Vergleich mit `mlp_baseline_v1` (CPU, 4 Agenten) zeigte, dass der Durchsatzgewinn primär von der Agentenanzahl stammt, nicht von der GPU – bei einem kleinen MLP ist die Unity-Simulation der Engpass, nicht die Netzberechnung. Einschränkend gilt: Dieser Lauf trainierte aufgrund des `Runtime Obstacles = 0`-Fehlers ohne Hindernisse und misst daher reine Zielnavigation.

8.2. Transformer- und LSTM-Iterationen (Milestone 7)

Die quantitative Diagnose der Iterationen V5–V14 (Entropy, Policy Loss, Value Loss, Reward, Episode Length) ist in Kapitel 7.6 dokumentiert. Das übergreifende Muster: erfolgreiches Lernen auf den Navigations-Phasen (Trivial bis TrivialHole, Erfolgsraten 62–79 %, in Einzelphasen bis 100 %), gefolgt von Stagnation an der Hazard-Schwelle. Die Diagnose identifizierte drei strukturelle Ursachen – den zu kurzen Planungshorizont ($\gamma = 0.99$ bei 1200-Step-Episoden macht das Ziel für frühe Aktionen unsichtbar), die PBRs-Pathologie (Luftlinien-Distanz

belohnt Stehenbleiben vor der Lava) und den Avoidance-Transfer aus der Hole-Phase (gelernte Gefahrenvermeidung verhindert die für das Sprunglernen nötige Lava-Annäherung).

Parallel wurde der eigenständige LSTM-Lauf `lstm_v2` dokumentiert: Nach Behebung von vier Patch-Bugs (u. a. fehlender Forward-Branch und ONNX-Export-Größe) erreichte er bei 650k Steps eine Erfolgsrate von 54 % auf Phase 1 mit stabilem Aufwärtstrend — der Nachweis, dass die Custom-LSTM-Integration funktioniert.

8.3. Paarweise Architektur-Vergleiche

8.3.1. Ray-Gruppe: LSTM (A3) vs. Transformer (A4)

Dieser Vergleich läuft aktuell als simultaner Drei-Architekturen-Lauf `model_comparison_v4` (Kapitel 7.7). Zwischenstand bei ca. 1,5 von 30 Mio. Steps (ca. 5 %, Curriculum-Phase 1 „TrivialLava,,): Über den bisherigen Verlauf zeigt der Transformer den konsistent besten Reward-Verlauf (letzter Checkpoint-Reward -3.8 gegenüber -4.9 bei LSTM und MLP), mit Peak-Erfolgsraten von ca. 48 % (Transformer) gegenüber ca. 44 % (LSTM) und ca. 42 % (MLP) in der Phase-0-Navigation. Eine erfolgreiche gezielte Lava-Überquerung wurde von keinem der drei Agenten bislang gezeigt (LavaCrossings = 0 über die gesamte Laufzeit); die Sprungversuchsrate liegt bei 28–49 % pro Episode.

TODO Endauswertung des Laufs `model_comparison_v4` bei Erreichen von 30 Mio. Steps ergänzen (ETA laut Trainingsrate: mehrere Tage Dauerbetrieb). Anschließend Wiederholungsläufe mit weiteren Seeds für die statistische Auswertung.

8.3.2. Kamera-Gruppe: LSTM (A1) vs. Transformer (A2)

TODO Geplant für Milestone 9 — CameraSensor-Integration und CNN-Encoder stehen noch aus.

8.3.3. Multi-Sensor-Gruppe: LSTM (A6) vs. Transformer (A5)

TODO Geplant für Milestone 10 — setzt Kamera-Gruppe voraus.

8.4. Generalisierung auf held-out Maps

TODO Der Generalisierungstest auf den drei unabhängigen Evaluations-Maps (Recursive Backtracking, Cellular Automata, Room-Placement) mit Erfolgsrate, Kollisionsrate und

sionsrate und Overfitting-Index pro Agent steht noch aus. Er setzt abgeschlossene Trainingsläufe voraus.

8.5. Statistische Auswertung

TODO Mann-Whitney-U-Tests mit Bonferroni-Korrektur, Cliff's Delta und Bootstrap-Konfidenzintervalle erfordern die 5-Seed-Wiederholungsläufe und stehen noch aus.

8.6. Diskussion

Auch ohne abgeschlossene Vergleichsmatrix lassen sich belastbare Zwischenbefunde festhalten. Erstens: Die Hypothesenbewertung ist noch offen, aber die bisherigen Daten deuten für die Ray-Gruppe auf einen moderaten Vorteil des Transformers in der Lerngeschwindigkeit hin – bei erheblich höherem Parameter- und Integrationsaufwand. Zweitens: Die dominanten Verhaltensmuster über alle Läufe sind konsistent. Wall-Hugging und Eck-Memorierung traten immer dann auf, wenn die Trainingsverteilung zu wenig Varianz bot; Lava-Avoidance erwies sich als hartnäckiges lokales Optimum, das durch die steile Curriculum-Stufe (Hole → Hazard) und die Reward-Asymmetrie stabilisiert wurde. Drittens: PBRs-Artefakte – die Belohnung der Luftlinien-Annäherung – erklärten mehrere scheinbar erfolgreiche Plateaus (V9, V12), in denen der Reward stieg, ohne dass die Erfolgsrate folgte. Als methodische Lehre wurde die Erfolgsrate (nicht der Reward) zur primären Fortschrittsmetrik erklärt.

9. Übertragbarkeit und praktische Anwendbarkeit

9.1. Analogie Labyrinth und reale Navigationsszenarien

Das Referenzszenario ist ein Serviceroboter (z. B. Kellnerroboter in einem Restaurant), der von der Küche zu einem Zieltisch navigiert, Hindernissen ausweicht und sich täglich an umgestellte Bestuhlung anpassen muss. Die Zuordnung: Korridore entsprechen Gängen zwischen Tischreihen, Wände fest verbauten Trennwänden, Lava und Löcher Stufen oder Kabeln am Boden, das prozedural wechselnde Layout der täglich veränderten Einrichtung, und das Episodenende einer abgeschlossenen Lieferung. Die Kernkompetenz, die das Labyrinth trainiert

— Generalisierung über nie gesehene Layouts statt Memorierung einer festen Karte — ist exakt die Anforderung solcher Einsatzumgebungen.

9.2. Hardware- und Software-Anforderungen

Konfiguration	Sensor-Hardware	Processing	Kosten
Ray-only (A3, A4)	Ultraschall / LiDAR	CPU-tauglich	niedrig
Kamera (A1, A2)	RGB-Kamera	GPU empfohlen	mittel
Multi-Sensor (A5, A6)	Kamera + LiDAR	GPU, hoher Datendurchsatz	hoch

Tabelle 6 — Hardware-Anforderungen je Sensor-Konfiguration

Softwareseitig unterscheidet sich die Inferenz-Latenz: Ein LSTM rechnet sequenziell in $O(n)$ mit niedriger Latenz pro Schritt, ein Transformer benötigt $O(\text{seq}^2)$ -Attention über das Kontextfenster — bei Echtzeitanforderungen (z. B. 10 Hz Navigationsrate) ist die Transformer-Latenz konkret zu messen.

9.3. Bewertungsmatrix

Die geplante Bewertungsmatrix gewichtet Erfolgsrate im Training (30 %), Generalisierung auf held-out Maps (25 %), Hardware-Kosten (20 %), Inferenz-Latenz (15 %) und Kollisionsrobustheit (10 %) über alle Agenten-Konfigurationen. Qualitativ zeichnen sich die Empfehlungsprofile bereits ab: Ray+LSTM für einfache, bekannte Strukturen mit Kostenlimit; Ray+Transformer für komplexe Routen mit vielen Abzweigungen; Kamera-Konfigurationen für visuell markierte Ziele; Multi-Sensor für kritische Umgebungen mit hoher Sicherheitsanforderung.

TODO Die empirische Befüllung der Bewertungsmatrix setzt die abgeschlossene Vergleichsmatrix (M11.1) voraus.

9.4. Limitationen der Übertragbarkeit

Die Übertragbarkeit ist durch mehrere Abstraktionen begrenzt: Das Labyrinth ist im Wesentlichen zweidimensional (keine Rampen, Höhenunterschiede oder wechselnde Bodenbeläge); alle Hindernisse sind statisch, während sich in realen Umgebungen Menschen bewegen; der Sim-to-Real-Gap zwischen Unity-Physik und realer Roboterphysik (Schlupf, Trägheit, Sensorlatenz) ist nicht adressiert; und die simulierte Sensorik ist idealisiert — reale Kameras und LiDARs haben Rauschen, Verzerrung und Reflexionen. Transferempfehlungen aus dieser Arbeit sind daher qualitativer Natur und nicht experimentell auf realer Hardware validiert.

10. Fazit und Ausblick

10.1. Zusammenfassung

Im Projektverlauf entstand eine vollständige experimentelle Plattform für den Vergleich von RL-Architekturen: eine 3D-Labyrinthwelt mit grid-basiertem Map-System, eine vollautomatische prozedurale Map-Generierung mit semantischer Lösbarkeitsvalidierung, ein achstufiges (später siebenstufiges) Curriculum-System, ein iterativ kalibriertes Reward-System mit PBRS und Curiosity, Custom-Policies für LSTM und Transformer über einen wartbaren venv-Patch, Multi-Area- und Multi-Instanz-Trainingsinfrastruktur mit bis zu 96 parallelen Agenten sowie eine Evaluations-Pipeline mit projektspezifischen TensorBoard-Metriken und automatisierten Analyse-Reports. Der laufende simultane Drei-Architekturen-Vergleich (MLP/LSTM/Transformer im selben Environment) stellt methodisch die bislang sauberste Vergleichsbasis des Projekts dar.

10.2. Beantwortung der Forschungsfragen

- **RQ1 (Sensortyp):** Noch offen — Kamera- und Multi-Sensor-Agenten stehen aus. Bisher wurde ausschließlich die Ray-Modalität untersucht.
- **RQ2 (Temporal-Architektur):** Teilweise beantwortbar. Beide Sequenzarchitekturen sind lauffähig integriert; der Transformer zeigt im laufenden Direktvergleich einen moderaten Vorsprung, erkauft durch einen rund 13-fachen Parameteraufwand des Speicher-Moduls und erheblich höheren Integrationsaufwand (PPO-Konsistenz, ONNX-Workarounds). Ein belastbares Urteil erfordert den abgeschlossenen 30M-Steps-Lauf und Mehrfach-Seeds.
- **RQ3 (Sensor-Fusion):** Noch offen — setzt die Kamera-Gruppe voraus.
- **RQ4 (Praktische Übertragbarkeit):** Qualitativ beantwortet (Kapitel 9): Ray-basierte Konfigurationen sind die kosteneffiziente Basis, die Architekturwahl folgt dem Komplexitätsprofil der Einsatzumgebung; eine empirische Bewertungsmatrix steht aus.

10.3. Limitationen und Lessons Learned

Vier übergreifende Erkenntnisse prägen das Projekt. **Erstens:** PPO und Transformer sind nicht trivial kombinierbar — die Konsistenz zwischen Inferenz- und Trainingssequenzen ist eine harte Korrektheitsanforderung, deren Verletzung nicht als Fehler, sondern als stilles

Nicht-Lernen sichtbar wird. **Zweitens:** PBRS ist nützlich, aber farmbar; die Wahl des Shaping-Gammas und der Skala entscheidet darüber, ob Annäherung oder Ankommen gelernt wird – und der Discount-Faktor bestimmt, ob das Ziel für frühe Aktionen überhaupt „sichtbar„ ist. **Drittens:** Curriculum Learning funktioniert, aber Phasenwechsel sind Stress-Tests für gelernte Policies; zu steile Stufen (Hole → Hazard) erzeugen unüberwindbare Wände, und gelernte Vermeidungsstrategien transferieren auch dorthin, wo sie schaden. **Viertens:** Reward Engineering ist Detailarbeit mit großer Hebelwirkung – und Konfigurationsmanagement (Prefab-Serialisierung vs. Code-Defaults, Persistenz von Trainingszustand über Neustarts) ist eine ebenso reale Fehlerquelle wie die Lernalgorithmik selbst.

10.4. Ausblick

Die nächsten Schritte sind klar priorisiert: Abschluss des laufenden 30M-Steps-Direktvergleichs und dessen Endauswertung; Wiederholungsläufe mit den verbleibenden Seeds für die statistische Absicherung; der Generalisierungstest auf den held-out Maps; anschließend der Kamera-Pfad (CNN-Encoder, M8–M9) und die Multi-Sensor-Fusion (M10) zur Vervollständigung der Agenten-Matrix. Darüber hinaus bieten sich als Erweiterungen dynamische Hindernisse, kooperative Multi-Agent-Szenarien und perspektivisch der Sim-to-Real-Transfer auf einen physischen Roboter an.

Anhang

11. Trainingskonfiguration (Direktvergleich)

Kernparameter der YAML-Konfiguration `config/model_comparison_standalone.yaml` für den simultanen Drei-Architekturen-Vergleich (identisch für alle drei Behaviors, nur memory unterscheidet sich):

Parameter	MLP	LSTM	Transformer
trainer_type	ppo	ppo	ppo
learning_rate	3.0e-4	3.0e-4	3.0e-4
batch_size	2048	2048	2048
buffer_size	102400	102400	102400
beta / epsilon / lambda	5e-3 / 0.2 / 0.95	5e-3 / 0.2 / 0.95	5e-3 / 0.2 / 0.95
num_epoch	5	5	5
hidden_units / num_layers	256 / 2	256 / 2	256 / 2
memory_type	—	lstm	transformer
sequence_length	—	32	128
memory_size	—	256	256
gamma (extrinsic)	0.995	0.995	0.995
curiosity strength	0.05	0.05	0.05
max_steps	30.000.000	30.000.000	30.000.000
time_horizon	512	512	512
checkpoint_interval	200.000	200.000	200.000

Umgebungseinstellungen: `num_envs: 2, seed: 42, Headless-Standalone-Build, --time-scale 20.`

12. Reward-Tabelle (Stand V13)

Ereignis	Reward	Vergeben in
Ziel erreicht	+30	OnTriggerEnter, Tag Goal
Lava-Tod	-3	OnTriggerEnter, Tag Lava
Hole-Tod	-3	OnTriggerEnter, Tag KillZone
Timeout (MaxStep)	-10	OnActionReceived bei StepCount $\geq$ MaxStep-1
Step-Penalty	-0.001 pro Step	OnActionReceived
PBRS-Shaping	$(\text{prevDist} - \gamma \cdot \text{currDist}) \cdot 0.005$	OnActionReceived
Lava-Überquerung	+8 (lavaCrossingReward)	Landung nach Flug über Lava
Wall-Climb	Penalty bei $y > \text{spawnY} + \text{wallClimbMaxY}$	OnActionReceived

MaxStep ist curriculum-abhängig (phaseMaxSteps): 600 Steps in den Trivial-Phasen bis 2500 Steps in Hard.

13. Curriculum-Phasen (Direktvergleich, Stand Juli 2026)

Phase	Schwierigkeit	Schwelle (Episoden)
0	Trivial (7×7 offener Raum)	500
1	TrivialLava (1-Zellen-Lava-Wand, Sprung erzwungen)	2000
2	TrivialCorr	1800
3	TrivialLava (Wiederholung)	1000
4	Easy	8000
5	Medium	12000
6	Hard	20000

Der Phasenfortschritt wird seit Juli 2026 pro Environment-Instanz atomar in JSON-Dateien persistiert (curriculum_state_<port>.json), sodass Trainer-Neustarts den Curriculum-Fortschritt nicht mehr zurücksetzen.

14. Bearbeitete GitHub-Issues (Übersicht)

Bereich	Issues
Milestones 1–2 (Map-System)	#2, #3, #4, #19, #20, #39–#44
Milestone 3 (Agent-Grundsystem)	#21–#36
Milestone 4 (Hindernisse & Todeslogik)	#82–#88, #105, #107
Milestone 5 (Reward & Training)	#93–#99
Milestone 6 (Prozedural & Curriculum)	#110–#112, #127, #133, #134
Milestone 7 (Transformer & LSTM)	#113–#115, #131
Geplant M8–M11	#116–#126

15. Hardware- und Software-Stack

Komponente	Version / Ausstattung
Unity Editor	2021.3 (Projektstart) / 6000.2.6f1 (Standalone-Builds)
ML-Agents Unity-Package	2.0.2
mlagents (Python)	0.30.0 (M5–M7) / 1.1.0 (Direktvergleich)
PyTorch	2.0.1+cu118 (M5–M7) / 2.3.1 (Direktvergleich)
Python	3.10 (venv)
TensorBoard	2.13.0
Trainings-Hardware (GPU)	NVIDIA RTX 3050 Laptop (4 GB VRAM), AMD Ryzen 5 5625U
Trainings-Hardware (CPU)	Surface-Gerät, 16 GB RAM (Direktvergleich, CPU-Training)

16. Repository-Struktur

```

KI_Agenten/
├── Assets/
│   ├── Scripts/
│   │   ├── Map/          (CellType, MapData, MapGenerator, RoomCorridorGraph,
│   │   │               ProceduralLayoutGenerator, ObstacleClusterPlacer,
│   │   │               SemanticPathfinder, CurriculumConfig,
CurriculumTracker, TilePool)
│   │   └── Agent/       (LabyrinthAgent)

```

```

|   |   └─ Camera/      (ThirdPersonCamera, ModelComparisonCameraController)
|   |   └─ Competition/(ScriptedAI-Agent – Messe-Demo)
|   └─ Editor/          (MapGeneratorEditor, StandaloneBuild,
CurriculumPhasePrepender)
|   └─ Layouts/          (statische + prozedural generierte MapData-Assets)
|   └─ Prefabs/          (Agent, Map-Tiles, Hindernisse)
└─ config/              (versionierte YAML-Trainingskonfigurationen)
└─ training/            (patch_mlagents.py, transformer_policy.py,
lstm_policy.py,
|                        hourly_analysis.py)
└─ results/             (Trainingsläufe: mlp_baseline_*, lstm_*, v5-v14,
|                        model_comparison_v1-v4)
└─ Dokumentation/      (technische Markdown-Dokumentation)
└─ Studienarbeit_Dokumentation/ (dieser Bericht, Typst)

```

Branch-Historie: Milestone-1,2 → Milestone-3-Agent-Grundsystem → milestone-4-hindernisse-und-interaktion → milestone-5-reward-system-training → milestone-7 (Transformer/LSTM-Integration, V5-V14) → Messe2 (Messe-Demo mit GUI und Scripted-AI-Vergleichsagent) → ModelTrainingComparsion (laufender Direktvergleich).

Literatur

- [1] A. Vaswani u. a., „Attention is All You Need“, in *Advances in Neural Information Processing Systems*, 2017.
- [2] S. Hochreiter und J. Schmidhuber, „Long Short-Term Memory“, *Neural Computation*, Bd. 9, Nr. 8, S. 1735–1780, 1997.
- [3] R. S. Sutton und A. G. Barto, *Reinforcement Learning: An Introduction*, 2. Aufl. Cambridge, MA: MIT Press, 2018.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, und O. Klimov, „Proximal Policy Optimization Algorithms“. 2017.
- [5] L. Chen u. a., „Decision Transformer: Reinforcement Learning via Sequence Modeling“, in *Advances in Neural Information Processing Systems*, 2021.
- [6] E. Parisotto u. a., „Stabilizing Transformers for Reinforcement Learning“, in *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- [7] A. Juliani u. a., „Unity: A General Platform for Intelligent Agents“. 2020.
- [8] V. Mnih u. a., „Human-level control through deep reinforcement learning“, *Nature*, Bd. 518, Nr. 7540, S. 529–533, 2015.
- [9] Y. Bengio, J. Louradour, R. Collobert, und J. Weston, „Curriculum Learning“, in *Proceedings of the 26th International Conference on Machine Learning*, 2009, S. 41–48.
- [10] N. Justesen, R. R. Torrado, P. Bontrager, A. Khalifa, J. Togelius, und S. Risi, „Illuminating Generalization in Deep Reinforcement Learning through Procedural Level Generation“, in *NeurIPS Workshop on Deep Reinforcement Learning*, 2018.
- [11] A. Y. Ng, D. Harada, und S. Russell, „Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping“, in *Proceedings of the 16th International Conference on Machine Learning*, 1999, S. 278–287.
- [12] D. Pathak, P. Agrawal, A. A. Efros, und T. Darrell, „Curiosity-driven Exploration by Self-supervised Prediction“, in *Proceedings of the 34th International Conference on Machine Learning*, 2017.

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Studienarbeit

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Friedrichshafen, 08.07.2026

Finn Ludwig, Ekaterina Karavaeva, David Pelcz, Alexander Bernecker